

INFORME DE DESPLIEGUE WEB

Mara Beauty Studio

Configuracion de Hosting Gratuito y Dominio Personalizado

Plataformas: GitHub · Netlify · Render.com · Freenom

Proyecto:	Mara Beauty Studio — Gestion de Citas
Materia:	Tecnologias Web / Despliegue de Aplicaciones
Autor:	Andres Velandia
Fecha:	Mayo 2026
Version:	1.0



TABLA DE CONTENIDOS

1. Introduccion

2. Descripcion del Proyecto

- > 2.1 Arquitectura del Sistema
- > 2.2 Estructura de Archivos
- > 2.3 Modelo de Datos y API

3. Seleccion de Plataformas

- > 3.1 Plataformas Seleccionadas y Justificacion
- > 3.2 Arquitectura de Despliegue

4. Preparacion del Repositorio en GitHub

- > 4.1 Creacion de Cuenta y Repositorio
- > 4.2 Configuracion del .gitignore
- > 4.3 Subida delCodigo

5. Despliegue del Frontend en Netlify

- > 5.1 Creacion de Cuenta
- > 5.2 Metodo 1: Drag & Drop
- > 5.3 Metodo 2: Despliegue desde GitHub
- > 5.4 Archivo netlify.toml
- > 5.5 Variables de Entorno y Dominio Personalizado

6. Despliegue del Backend en Render.com

- > 6.1 Creacion de Cuenta
- > 6.2 Creacion de la Base de Datos PostgreSQL
- > 6.3 Creacion del Web Service
- > 6.4 Variables de Entorno y Verificacion

7. Configuracion del Dominio en Freenom

- > 7.1 Registro del Dominio Gratuito
- > 7.2 Configuracion DNS
- > 7.3 SSL/HTTPS Automatico

8. Configuracion del Frontend para Produccion

9. Verificacion del Despliegue

1. INTRODUCCION

El presente informe documenta el proceso completo de despliegue de la aplicacion web **Mara Beauty Studio** en plataformas de hosting y dominio gratuitas disponibles en internet. El objetivo es hacer accesible la aplicacion a traves de una direccion web publica, utilizando exclusivamente servicios gratuitos, lo que permite demostrar la viabilidad de publicar proyectos web profesionales sin costo inicial.

1.1 Objetivo del Informe

Documentar de manera detallada, con comandos reales, el proceso de:

- Selecccion justificada de plataformas gratuitas de hosting y dominio.
- Preparacion del codigo fuente de Mara Beauty Studio para produccion.
- Despliegue del frontend estatico en Netlify.
- Despliegue del backend Node.js y la base de datos PostgreSQL en Render.com.
- Obtencion y configuracion de un dominio personalizado gratuito mediante Freenom.
- Verificacion del funcionamiento completo de la aplicacion en produccion.

1.2 Descripcion del Proyecto

Mara Beauty Studio es una aplicacion web completa de gestion de citas para un salon de belleza. Permite a los clientes visualizar los horarios disponibles y reservar citas en linea, mientras que el administrador del salon puede crear, gestionar y eliminar los slots de atencion desde un panel de control protegido con autenticacion JWT. El sistema tambien envia correos electronicos de confirmacion y cancelacion a los clientes mediante Nodemailer.

1.3 Alcance del Despliegue

El despliegue comprende tanto la capa frontend (archivos estaticos HTML/CSS/JS) como la capa backend (servidor Node.js + API REST) y la base de datos relacional (PostgreSQL). Ambas capas quedaran conectadas y accesibles publicamente a traves de un dominio personalizado gratuito.

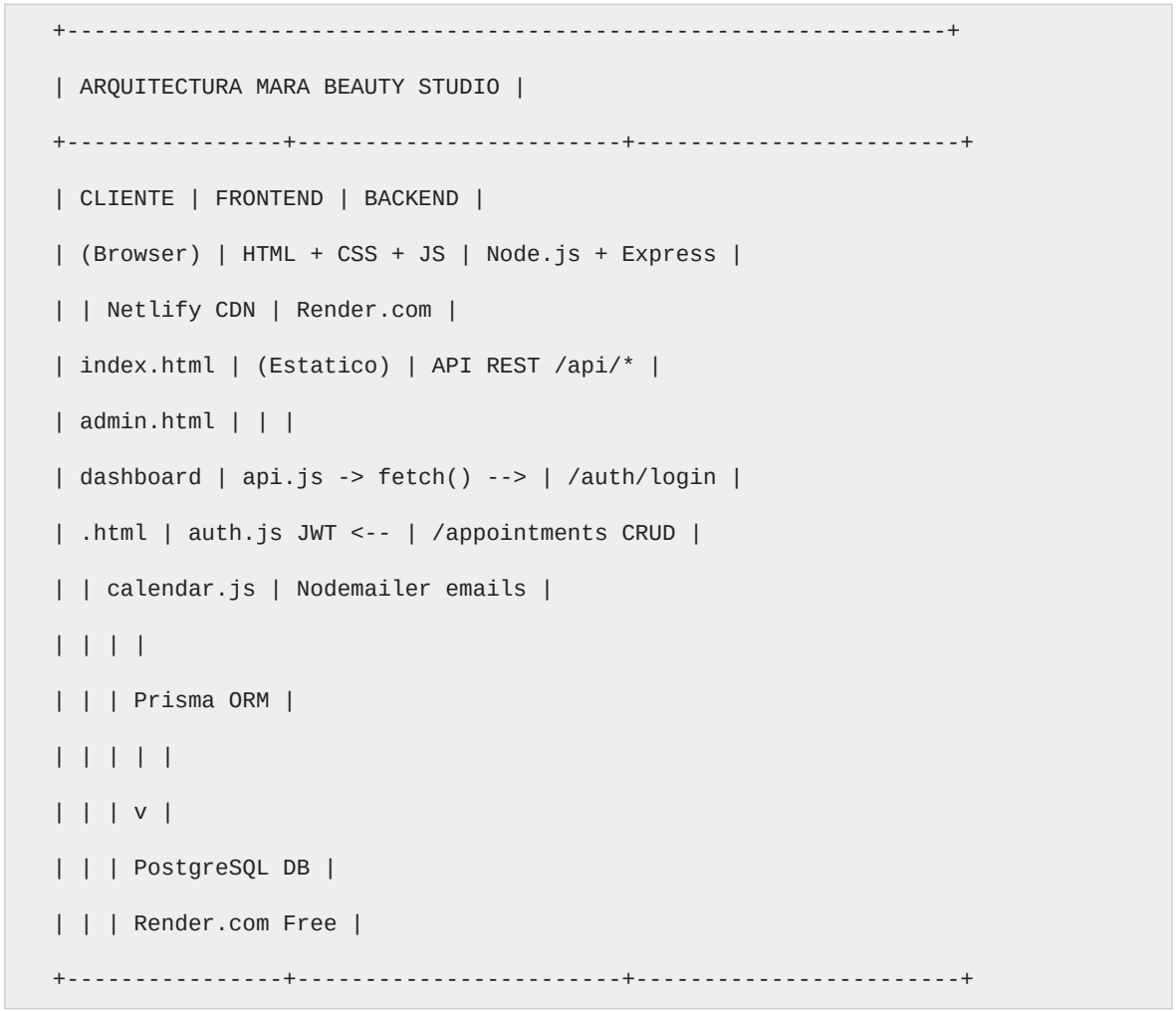
Nota: Este informe fue elaborado con fines academicos y utiliza planes gratuitos de cada plataforma. Para un ambiente de produccion empresarial se recomienda migrar a planes pagos con SLA garantizado.

2. DESCRIPCION DEL PROYECTO

2.1 Arquitectura del Sistema

Mara Beauty Studio sigue una arquitectura **cliente-servidor desacoplada**. El frontend es un sitio web estatico que se comunica con el backend a traves de una API REST. La base de datos es administrada exclusivamente por el backend mediante el ORM Prisma.

Figura 1 - Diagrama de Arquitectura del Sistema



2.2 Estructura de Archivos del Proyecto

A continuacion se describe cada archivo del proyecto y su funcion dentro del sistema:

Archivo / Directorio	Capa	Descripcion
frontend/index.html	Frontend	Vista principal del cliente: muestra el calendario semanal con slots disponibles y permite reservar citas

Archivo / Directorio	Capa	Descripcion
frontend/admin.html	Frontend	Pagina de login del administrador con formulario de autenticacion JWT
frontend/dashboard.html	Frontend	Panel de control del admin: crear slots de atencion, ver y eliminar citas existentes
frontend/simulador-correo.html	Frontend	Vista de prueba que simula los correos de confirmacion y cancelacion
frontend/css/style.css	Frontend	Hoja de estilos global: paleta de colores rosada, fuente Poppins, disenio responsive
frontend/js/api.js	Frontend	Modulo central de comunicacion: define BASE_URL y funciones fetch() para todos los endpoints
frontend/js/auth.js	Frontend	Manejo de autenticacion: almacena/lee el JWT en localStorage, protege rutas admin
frontend/js/calendar.js	Frontend	Logica del calendario: navegacion por semanas, renderizado de slots, seleccion de citas
frontend/js/client.js	Frontend	Orquesta las interacciones del cliente: reservar y cancelar citas, mostrar modales
frontend/js/dashboard.js	Frontend	Logica del dashboard admin: crear slots, listar y eliminar appointments via API
frontend/js/ui.js	Frontend	Manipulacion del DOM: mostrar alertas, abrir/cerrar modales, actualizar vistas
frontend/js/utils.js	Frontend	Utilidades: formateo de fechas en espanol (Bogota), validaciones de formularios
backend/server.js	Backend	Punto de entrada del servidor: carga dotenv, instancia Express y arranca en PORT
backend/src/app.js	Backend	Configura middlewares (CORS, JSON parser), monta rutas /api/auth y /api/appointments
backend/prisma/schema.prisma	Backend/DB	Esquema Prisma: define modelos User, Client y Appointment con sus relaciones
backend/prisma/seed.js	Backend/DB	Script de seed: crea el usuario administrador inicial con contrasena hasheada en bcrypt
backend/package.json	Backend	Manifiesto del proyecto: dependencias, scripts npm (start, dev, db:migrate, db:seed)

Tabla 1 - Estructura de archivos del proyecto Mara Beauty Studio

2.3 Modelo de Datos y Endpoints de la API

El backend expone los siguientes endpoints REST:

Metodo	Ruta	Auth	Descripcion
POST	/api/auth/login	No	Login administrador. Retorna JWT token de acceso
GET	/api/appointments?week=X&year;=Y	No	Lista todas las citas de la semana indicada
POST	/api/appointments	JWT	Crea uno o varios slots de atencion (solo admin)
DELETE	/api/appointments/:id	JWT	Elimina un slot (disponible o reservado) solo admin
POST	/api/appointments/:id/book	No	Reserva una cita: recibe nombre y email del cliente
POST	/api/appointments/:id/cancel	No	Cancela una cita reservada y libera el slot

Tabla 2 - Endpoints de la API REST de Mara Beauty Studio

3. SELECCION DE PLATAFORMAS

3.1 Plataformas Seleccionadas y Justificacion

NETLIFY - Hosting del Frontend

Seleccionado para alojar los archivos estaticos del frontend (HTML, CSS, JS). Ofrece despliegue continuo desde GitHub, CDN global para carga rapida, certificado SSL/HTTPS automatico gratuito via Let's Encrypt y soporte para dominios personalizados sin costo adicional. El plan Starter gratuito incluye 100 GB de bandwidth mensual, suficiente para el volumen de un salon de belleza.

RENDER.COM - Backend + Base de Datos

Seleccionado para alojar el servidor Node.js/Express y la base de datos PostgreSQL. Render ofrece un Web Service gratuito para aplicaciones Node.js con despliegue automatico desde GitHub y una instancia de PostgreSQL en el plan Free. Soporta variables de entorno, comandos de build personalizados (incluyendo migraciones Prisma) y proporciona una URL publica bajo el subdominio .onrender.com.

FREENOM - Dominio Personalizado Gratuito

Seleccionado para obtener un dominio personalizado gratuito. Freenom ofrece dominios con extensiones .tk, .ml, .ga, .cf y .gq sin costo durante los primeros 12 meses. Los registros DNS de Freenom se configuran para apuntar al CDN de Netlify, donde el frontend es servido. El dominio seleccionado es: **mara-beauty-studio.tk**

3.2 Diagrama de Arquitectura de Despliegue

El siguiente diagrama muestra el flujo completo desde que el usuario accede al dominio hasta que la informacion es procesada por la base de datos en produccion:

Figura 2 - Flujo de Arquitectura de Despliegue

```
+-----+ DNS lookup +-----+
| USUARIO | -----> | FREenom DNS |
| (Browser) | | mara-beauty- |
| | <----- | studio.tk |
+-----+ IP de Netlify +-----+
|
| HTTPS Request
v
```

```
+-----+
| NETLIFY CDN |
| (Frontend Estatico) |
| index.html / admin.html |
| dashboard.html / style.css |
| js/api.js -> BASE_URL=Render |
+-----+

|
| fetch() API calls (HTTPS)
v
+-----+

| RENDER.COM |
| Web Service (Node.js) |
| mara-beauty-api.onrender.com |
| Express + Prisma ORM |
| JWT Auth + Nodemailer |
+-----+

|
| Prisma queries
v
+-----+

| RENDER.COM - PostgreSQL DB |
| marabeauty-db (Free tier) |
| Tablas: User, Client, |
| Appointment |
+-----+
```

Fuente: Elaboracion propia basada en la documentacion de Netlify y Render.com

4. PASO A PASO: REPOSITORIO EN GITHUB

GitHub actua como el repositorio central del codigo fuente. Tanto Netlify como Render.com se conectan a el para obtener el codigo y desplegar automaticamente cada vez que se realice un push a la rama principal.

Acceder a GitHub: <https://github.com>

4.1 Crear Cuenta y Repositorio en GitHub

Paso 1 - Registro: Ingresar a <https://github.com/signup> y crear una cuenta gratuita con email, usuario y contrasena.

Paso 2 - Nuevo repositorio: Hacer clic en el boton verde 'New' en la pagina principal o ir a <https://github.com/new>

Paso 3 - Configuracion: Asignar nombre: mara-beauty-studio, seleccionar visibilidad Public, marcar 'Add a README file' y hacer clic en 'Create repository'.

Paso 4 - URL del repositorio: Se genera automaticamente la URL:
[https://github.com/\[usuario\]/mara-beauty-studio](https://github.com/[usuario]/mara-beauty-studio)

4.2 Configuracion del Archivo .gitignore

Antes de subir el codigo, se debe crear un archivo **.gitignore** en la raiz del proyecto para excluir archivos sensibles y directorios innecesarios:

Archivo: .gitignore

```
# Dependencias
node_modules/

# Variables de entorno (NUNCA subir al repositorio)
.env
.env.local
.env.production

# Base de datos local SQLite de Prisma
prisma/dev.db
prisma/dev.db-journal

# Archivos del sistema operativo
.DS_Store
Thumbs.db
```

```
# Logs

*.log

npm-debug.log*
```

Nota: El archivo `.env` contiene el `JWT_SECRET` y `DATABASE_URL` con credenciales reales. Jamás debe subirse al repositorio público. Las variables se configuran directamente en los paneles de Netlify y Render.com.

4.3 Subida del Código a GitHub

Ejecutar los siguientes comandos en la terminal desde la raíz del proyecto:

Terminal - Comandos Git para subir el proyecto

```
# 1. Inicializar git en el directorio del proyecto

git init

# 2. Agregar todos los archivos al staging (excepto los del .gitignore)

git add .

# 3. Crear el primer commit

git commit -m "Initial commit - Mara Beauty Studio"

# 4. Renombrar la rama principal a 'main'

git branch -M main

# 5. Conectar con el repositorio remoto en GitHub

git remote add origin https://github.com/[usuario]/mara-beauty-studio.git

# 6. Subir el código a GitHub

git push -u origin main
```

Tras ejecutar estos comandos, el código de Mara Beauty Studio queda disponible en GitHub y listo para ser enlazado con Netlify y Render.com en los siguientes pasos.

5. PASO A PASO: DESPLIEGUE DEL FRONTEND EN NETLIFY

Netlify es una plataforma de hosting especializada en sitios web estaticos con CDN global. Es ideal para el frontend de Mara Beauty Studio porque los archivos HTML, CSS y JS no requieren servidor propio para ser servidos.

Acceder a Netlify: <https://www.netlify.com>

5.1 Crear Cuenta en Netlify

- Ingresar a <https://app.netlify.com/signup>
- Seleccionar 'Sign up with GitHub' para vincular directamente la cuenta de GitHub
- Autorizar a Netlify el acceso a los repositorios
- El plan Starter (gratuito) se activa automaticamente al registrarse

5.2 Metodo 1: Despliegue por Arrastre (Drag & Drop)

Este metodo es el mas rapido y no requiere configuracion adicional:

Paso 1. Abrir en el navegador: <https://app.netlify.com/drop>

Paso 2. En el explorador de archivos del computador, abrir la carpeta del proyecto.

Paso 3. Seleccionar la carpeta 'frontend/' completa (que contiene index.html, admin.html, css/ y js/).

Paso 4. Arrastrar la carpeta 'frontend/' y soltarla en el area de arrastre de Netlify.

Paso 5. Netlify procesa los archivos en segundos y genera una URL automatica como: <https://amazing-name-123456.netlify.app>

Paso 6. El sitio ya es accesible publicamente desde esa URL.

5.3 Metodo 2: Despliegue Continuo desde GitHub (Recomendado)

Este metodo configura el despliegue automatico: cada vez que se haga push al repositorio, Netlify redespiegara el sitio automaticamente:

Paso 1. En el dashboard de Netlify, hacer clic en 'Add new site' -> 'Import an existing project'.

Paso 2. Seleccionar 'Deploy with GitHub'.

Paso 3. Buscar y seleccionar el repositorio 'mara-beauty-studio'.

Paso 4. En 'Build settings' configurar: Branch to deploy: main | Base directory: frontend | Build command: (dejar vacio para sitio estatico) | Publish directory: frontend

Paso 5. Hacer clic en 'Deploy site'. Netlify clona el repositorio y despliega los archivos.

Paso 6. En aproximadamente 30-60 segundos, el sitio queda publicado con una URL .netlify.app

5.4 Archivo de Configuración netlify.toml

Para optimizar el despliegue, se crea el archivo **netlify.toml** en la raíz del proyecto:

Archivo: netlify.toml

```
[build]

publish = "frontend"

# Redirigir todas las rutas al index.html

[[redirects]]

from = "/*"

to = "/index.html"

status = 200

# Headers de seguridad

[[headers]]

for = "/*"

[headers.values]

X-Frame-Options = "DENY"

X-Content-Type-Options = "nosniff"

Referrer-Policy = "strict-origin-when-cross-origin"
```

5.5 Variables de Entorno y Dominio Personalizado

Configurar variable de entorno en Netlify:

- Ir a: Site settings -> Environment variables -> Add a variable
- Key: API_BASE_URL | Value: https://mara-beauty-api.onrender.com
- Hacer clic en 'Save' y redespargar el sitio

Configurar dominio personalizado (una vez obtenido en Freenom - ver Sección 7):

- Site settings -> Domain management -> Add a domain alias
- Ingresar: mara-beauty-studio.tk -> Verify
- SSL/HTTPS se provisiona automáticamente (Let's Encrypt) en 5-10 minutos

Elemento	Valor
URL Netlify (automatica)	https://mara-beauty-studio.netlify.app

URL con dominio personalizado	https://mara-beauty-studio.tk
SSL/HTTPS	Automatico via Let's Encrypt (gratis)
CDN	Global (USA, Europa, Asia)
Bandwidth gratuito	100 GB / mes
Tiempo de build (estatico)	< 30 segundos

Tabla 3 - Resumen de configuracion Netlify para Mara Beauty Studio

6. PASO A PASO: DESPLIEGUE DEL BACKEND EN RENDER.COM

Render.com es la plataforma seleccionada para alojar el servidor Node.js/Express de Mara Beauty Studio y la base de datos PostgreSQL. Su plan gratuito incluye un Web Service para aplicaciones Node.js y una base de datos PostgreSQL, ambos necesarios para el backend.

Acceder a Render.com: <https://render.com>

6.1 Crear Cuenta en Render.com

- Ingresar a <https://dashboard.render.com/register>
- Registrarse con GitHub (recomendado, ya que simplifica la conexión con el repositorio)
- Confirmar el correo electrónico
- El plan Free se activa automáticamente

6.2 Crear la Base de Datos PostgreSQL

El primer recurso a crear es la base de datos, ya que se necesita su URL de conexión para configurar el Web Service:

Paso 1. En el dashboard de Render, hacer clic en 'New +' -> 'PostgreSQL'.

Paso 2. Configurar: Name: marabeauty-db | Database: marabeauty | User: marabeauty_user | Region: Oregon (US West) | Plan: Free.

Paso 3. Hacer clic en 'Create Database'. El proceso tarda 1-2 minutos.

Paso 4. Una vez creada, Render muestra las credenciales. Copiar la 'Internal Database URL' (formato: postgresql://user:pass@host/db)

Paso 5. Anotar también la 'External Database URL' (para conectarse desde herramientas locales como DBeaver).

Ejemplo de Database URL generada por Render.com

```
# Formato de la Internal Database URL que proporciona Render:
postgresql://marabeauty_user:AbCdEfGhIjKl@dpg-xxxxx-a/marabeauty

# Esta URL se usará como valor de DATABASE_URL en las variables de entorno
```

6.3 Crear el Web Service (Backend Node.js)

Paso 1. En el dashboard de Render, hacer clic en 'New +' -> 'Web Service'.

Paso 2. Seleccionar 'Build and deploy from a Git repository' -> conectar con GitHub -> buscar 'mara-beauty-studio'.

Paso 3. Configurar los parametros del servicio:

Parametro	Valor
Name	mara-beauty-api
Root Directory	backend
Environment	Node
Region	Oregon (US West)
Branch	main
Build Command	npm install && npx prisma generate && npx prisma migrate deploy
Start Command	npm start
Plan	Free

Tabla 4 - Configuración del Web Service en Render.com

Paso 4. Hacer clic en 'Create Web Service'. Render clona el repositorio y ejecuta el Build Command.

Paso 5. Una vez completado el deploy, se genera la URL del API:
<https://mara-beauty-api.onrender.com>

6.4 Variables de Entorno y Verificación del API

Las variables de entorno reemplazan el archivo .env local. Se configuran en: Dashboard -> mara-beauty-api -> Environment -> Environment Variables:

Variable	Valor / Descripción
DATABASE_URL	postgresql://... (Internal DB URL del paso 6.2)
JWT_SECRET	Cadena aleatoria segura (ej: openssl rand -base64 32)
EMAIL_USER	correo@gmail.com (cuenta de envío Nodemailer)
EMAIL_PASS	Contraseña de aplicación de Gmail
NODE_ENV	production
PORT	10000

Tabla 5 - Variables de entorno configuradas en Render.com

Ejecutar seed inicial desde Render Shell y verificar el API:

Verificación de endpoints

```
# Desde Render Dashboard -> mara-beauty-api -> Shell:

node prisma/seed.js

# Test: verificar que el servidor responde
```

GET https://mara-beauty-api.onrender.com/api/appointments

Respuesta esperada: [] (array vacio de citas)

Test: verificar login admin

POST https://mara-beauty-api.onrender.com/api/auth/login

Body: { "username": "admin", "password": "admin123" }

Respuesta esperada: { "token": "eyJhbG..." }

Nota: En el plan gratuito de Render, el Web Service entra en modo 'sleep' despues de 15 minutos de inactividad. La primera request despues del sleep puede tardar 20-30 segundos en responder mientras el servicio se reactiva (cold start). Esto es normal en el plan Free.

7. PASO A PASO: DOMINIO GRATUITO EN FREENOM

Freenom es un proveedor de dominios que ofrece extensiones gratuitas: .tk, .ml, .ga, .cf y .gq. El dominio seleccionado para Mara Beauty Studio es **mara-beauty-studio.tk**. Los dominios .tk son propiedad de Tokelau (territorio de Nueva Zelanda) y se ofrecen gratuitamente como parte de un acuerdo de ingresos compartidos.

Acceder a Freenom: <https://www.freenom.com>

7.1 Registro del Dominio Gratuito

Paso 1. Ingresar a <https://www.freenom.com/es/index.html>

Paso 2. En el campo de búsqueda, escribir 'mara-beauty-studio' y presionar 'Check Availability'.

Paso 3. Freenom muestra las extensiones disponibles. Seleccionar '.tk' (gratuita por 12 meses) haciendo clic en 'Get it now!'.

Paso 4. Hacer clic en 'Checkout'. En la pagina de orden, cambiar el periodo a '12 Months @ FREE' y hacer clic en 'Continue'.

Paso 5. Si no se tiene cuenta, seleccionar 'I don't have an account' y registrarse con email.

Paso 6. Completar la informacion de contacto (nombre, direccion). Hacer clic en 'Complete Order'.

Paso 7. Revisar el correo electronico: Freenom enviara un email de verificacion. Hacer clic en el enlace para activar la cuenta.

Paso 8. Ingresar al panel de Freenom con las credenciales creadas. En 'My Domains' aparecera el dominio mara-beauty-studio.tk.

7.2 Configuracion de los Registros DNS

Los registros DNS indican a los navegadores a que servidor deben dirigirse cuando alguien escribe mara-beauty-studio.tk en la barra de direcciones:

- En el panel de Freenom, ir a: Services -> My Domains -> Manage Domain.
- Hacer clic en la pestana 'Manage Freenom DNS'.
- Agregar los siguientes registros DNS:

Tipo	Name (Host)	Target (Valor)	TTL	Proposito
A	@	104.198.14.52	3600	Apunta el dominio raiz a la IP de Netlify
CNAME	www	mara-beauty-studio.netlify.app	3600	Redirige www al subdominio de Netlify

Tabla 6 - Registros DNS a configurar en Freenom para apuntar a Netlify

Despues de guardar los registros DNS, ir al panel de Netlify:

- En Netlify: Site settings -> Domain management -> Add a domain alias
- Escribir: mara-beauty-studio.tk y hacer clic en 'Add domain alias'
- Hacer clic en 'Provision certificate' para activar HTTPS/SSL automatico

7.3 SSL/HTTPS Automatico con Let's Encrypt

Netlify se asocia con **Let's Encrypt** para emitir certificados SSL/TLS gratuitos de manera automatica. Una vez que los DNS propagan correctamente, Netlify detecta el dominio y emite el certificado en 5-15 minutos. El resultado es que el sitio queda accesible mediante HTTPS con el candado de seguridad visible en el navegador.

Nota: La propagacion DNS puede tomar entre 1 y 48 horas. Para verificar la propagacion se puede usar:
<https://www.whatsmydns.net>

8. CONFIGURACION DEL FRONTEND PARA PRODUCCION

Antes de hacer el despliegue final, se realizan ajustes en el código del frontend para que apunte al backend en producción (Render.com) en lugar de localhost.

8.1 Actualizar la URL del API en api.js

El archivo **frontend/js/api.js** contiene la URL base para todas las llamadas al API. Se debe cambiar de localhost a la URL de Render:

frontend/js/api.js - ANTES (local)

```
// ANTES (desarrollo local)

const BASE_URL = 'http://localhost:3000';
```

frontend/js/api.js - DESPUES (produccion)

```
// DESPUES (produccion en Render.com)

const BASE_URL = 'https://mara-beauty-api.onrender.com';
```

8.2 Configurar CORS en el Backend

El backend debe permitir peticiones HTTP desde el dominio de Netlify. Se configura CORS en **backend/src/app.js**:

backend/src/app.js - Configuración CORS para producción

```
const corsOptions = {
  origin: [
    'https://mara-beauty-studio.netlify.app',
    'https://mara-beauty-studio.tk',
    'https://www.mara-beauty-studio.tk',
    // Mantener localhost para desarrollo local:
    'http://localhost:5500',
    'http://127.0.0.1:5500',
  ],
  methods: ['GET', 'POST', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization'],
};

app.use(cors(corsOptions));
```

8.3 Commit y Push de los Cambios

Terminal - Push para redespliegue automatico

```
# Guardar los cambios

git add frontend/js/api.js backend/src/app.js

git commit -m "config: update API URL and CORS for production"

git push origin main

# Al hacer push:

# - Netlify detecta el cambio y redespliega el frontend automaticamente

# - Render.com detecta el cambio y redespliega el backend automaticamente
```

9. VERIFICACION DEL DESPLIEGUE

Una vez completados todos los pasos anteriores, se realiza una verificación integral del sistema para confirmar que todos los componentes funcionan correctamente en producción.

9.1 Lista de Verificación

OK	Frontend accesible en URL de Netlify https://mara-beauty-studio.netlify.app carga la página principal del cliente
OK	Dominio personalizado activo https://mara-beauty-studio.tk redirige correctamente al frontend
OK	SSL/HTTPS activo Candado de seguridad verde visible en la barra del navegador
OK	API responde correctamente GET https://mara-beauty-api.onrender.com/api/appointments retorna JSON
OK	Calendario muestra citas disponibles La vista del cliente muestra el calendario semanal con slots
OK	Flujo de reserva de cita funciona Cliente puede seleccionar slot, ingresar datos y confirmar la reserva
OK	Panel de administrador accesible https://mara-beauty-studio.tk/admin.html muestra el formulario de login
OK	Login JWT del administrador funciona Admin puede ingresar con credenciales y acceder al dashboard
OK	Dashboard admin funciona Admin puede crear y eliminar slots de atención desde dashboard.html
OK	Emails de confirmación se envían Al reservar una cita, el cliente recibe un correo de confirmación vía Nodemailer

9.2 URLs Finales del Sistema en Producción

Componente	URL de Producción	Descripción
Frontend (cliente)	https://mara-beauty-studio.tk	Página principal - reservar citas

Componente	URL de Produccion	Descripcion
Frontend (Netlify)	https://mara-beauty-studio.netlify.app	URL alternativa de Netlify
Admin Login	https://mara-beauty-studio.tk/admin.html	Acceso panel administrador
Dashboard Admin	https://mara-beauty-studio.tk/dashboard.html	Gestion de slots y citas
API REST	https://mara-beauty-api.onrender.com	Base URL del backend
API Appointments	https://mara-beauty-api.onrender.com/api/appointments	Endpoint principal de citas
API Login	https://mara-beauty-api.onrender.com/api/auth/login	Endpoint de autenticacion

Tabla 7 - URLs definitivas del sistema Mara Beauty Studio en produccion